

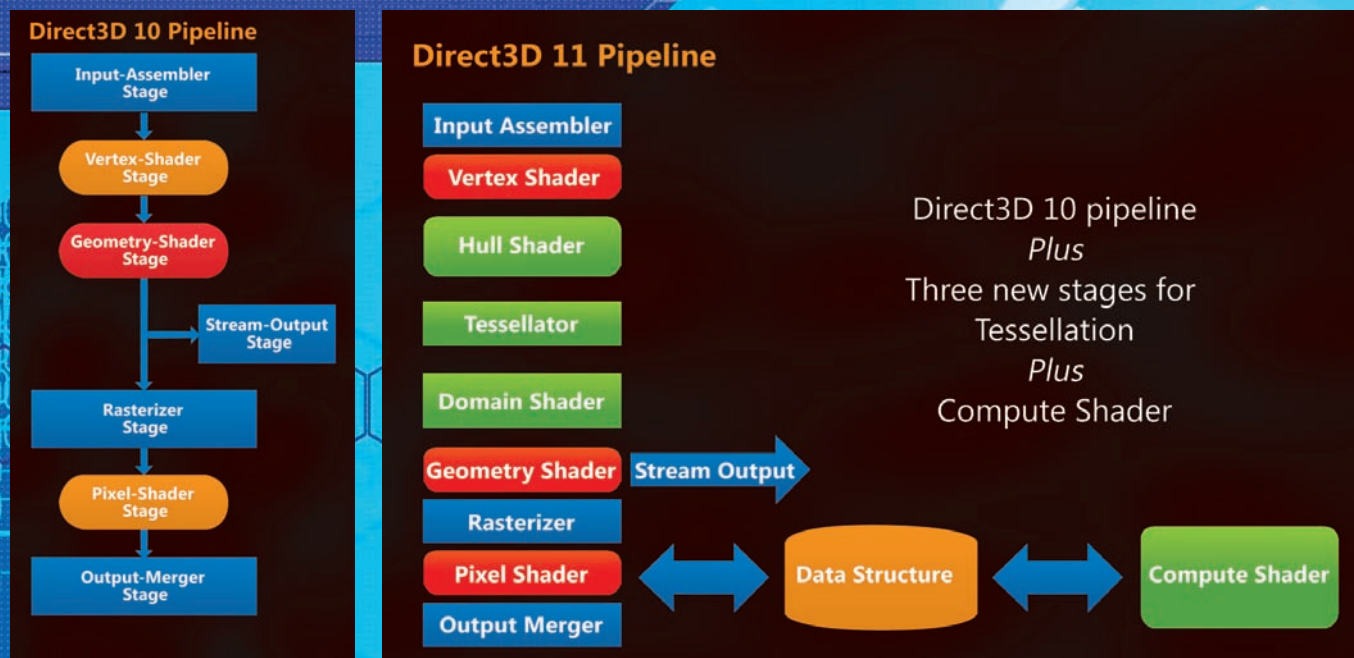
Digit Mini

As a special offer, new subscriptions this month will get a complimentary copy of Digit Mini

Toshiba Rezga

Toshiba has announced a launch of their 3D Rezga HDTVs in a couple of months

How stuff works



The DX Pipeline: I've got new stuff

The major changes to the DirectX pipeline can be easily see through the graphic representation above.

The additions highlighted on the right, in green, are the Tessellator, Hull Shader, Domain Shader and the Compute Shader.

Shader Model 5.0: Check out my new shade(r)s!

DX10 opened up the concept of programmability of a GPU. DX9 had only scraped the surface. DX10 was able to go a lot deeper – making it more than just a fixed-function vehicle to shuttle data.

One of the newbie's on showcase with DX11 is Shader Model 5.0. Incremental numbers mean an improved feature set as SM 5.0 is a superset of SM 4.1 (DX 10.1). When a game programmer codes for a game, he can choose from between these shader models – an older standard would maximise compatibility, while a newer one would improve visual quality and effects and improve performance and/or optimise resource usage. This is possible because essentially, the code remains unchanged. DX10 code is essentially DX11 code sans some advanced features. Thus DX11 will run on older (read DX10) hardware but DX11-specific

features will be unavailable. This gives developers flexibility because one set of code can target both DX10 and DX11 APIs and hardware, unless of course they wish to target a DX11-specific feature, in which case separate code will need to be used. On the whole however, this makes developer's lives easier.

If you look at the table below, you can see there are a few significant changes – like the move to double precision floating point operations, which, until now, was the sole realm of the CPU. This is in accordance with the IEEE-754 standard and many feel that this is yet another step forward for the GPGPU concept – after all double precision

SM 5.0 and SM 4.0 differences (Source AMD)

Feature	SM 4.0	SM 5.0	Benefits
Thread Dispatch	2D	3D	Replace multiple 2D thread arrays with a single 3D array
Thread Limit	768	1024	Execute up to 33% more simultaneous threads for better performance
Thread Group Shared Memory	16 kB	32 kB	Double the memory available for inter-thread communication
Shared Memory Access	256 byte write only	Full 32 kB read/write region	More efficient shared memory I/O
Atomic Operations	Not supported	Supported	Allows each thread to operate on protected memory locations – greatly simplifies implementation of CPU-based algorithms on GPU
Double Precision	Not supported	Supported	Allows 64-bit floating point precision for computations
Append/Consume Buffers	Not supported	Supported	Useful for building and accessing data in list or stack form
Unordered Access Views Bound to Compute Shader	1	8	Allows each compute shader to access multiple buffers for more flexibility
Unordered Access Views Bound to Pixel Shader	Not supported	8	Allows pixel shaders to access compute shader data for interoperability with rendering pipeline
Gather4	Not supported	Supported	Fetch adjacent data values from memory up to 4x faster